

Build Your Own DPF Monitor

A point-and-click guide to flashing the CYD 3.5" touchscreen board that watches your diesel particulate filter — written for complete beginners, no programming or command-line experience assumed.

For the 2014 Hyundai ix35 1.7 CRDi

Windows · Mac · Linux

With or without an AI assistant

IN THIS GUIDE

- What you're building
- Two ways to do this
- Shopping list
- Setting up your WiFi & Telegram secrets
- Formatting the SD card
- Checking it's actually working
- Windows — install & flash
- Mac — install & flash
- Linux — install & flash

01 What you're building

A small touchscreen box that plugs into your car's OBD-II port (the diagnostic socket, usually under the dashboard near the steering column — the same one a garage's scanner plugs into) via a small Bluetooth adapter. It reads how much soot is in your diesel particulate filter and warns you before it clogs. It doesn't change anything about how your car runs — it only reads data the engine computer already has.

It does **not** need your phone once it's working, and you do **not** need to pair anything in your computer's or phone's Bluetooth settings — the board talks to the adapter directly, on its own.

02 Two ways to do this

Option A — have an AI assistant walk you through it. Open [Claude](#) or [ChatGPT](#) in a browser tab and paste the box below. It will ask you questions and give you one step at a time, checking in before anything that matters.

```
I'm a complete beginner and I want to build a DPF (diesel particulate filter)
monitor for my car from an open-source project, using a ready-made ESP32-S3
```

touchscreen board called "CYD 3.5-inch". I have no programming or command-line experience. Walk me through this from scratch, GUI only - no terminal commands unless there is truly no other way, and if there isn't, explain what the command does in plain English before I type it.

The steps are documented at <repo-url>/docs/easy-setup/ (the hub page for the shopping list and config walkthrough, plus a windows.md / mac.md / linux.md page for installing software and flashing). Read those first - ask me to paste them if you can't open the link. Then ask which operating system I'm on and follow only that page.

Rules for you:

- One step at a time. After each step, tell me exactly what I should see if it worked, and wait for me to confirm before the next one.
- Never ask me to paste my WiFi password, Telegram bot token, or any other secret into this chat. Tell me which file to put it in on my own computer instead, and exactly what line to change.
- If something doesn't look like the guide says, stop and help me figure out why rather than telling me to skip it.
- Tell me plainly if something isn't possible rather than guessing.

I'll tell you when I'm ready for step one.

Option B — do it yourself, no AI. Follow the written steps in this guide, then jump to your operating system's section: Windows, Mac or Linux, later in this document.

Both options use exactly the same underlying steps — Option A just has something explaining each one to you as you go.

03 Shopping list

1. **The CYD 3.5" board** — sold on Amazon/AliExpress as "CYD ESP32 S3 Display 3.5 Inch IPS Capacitive Touch 320x480 ST77922 HMI Module, XiaoZhiAI" (around £20–25). Most listings include a USB-C cable, but see the cable warning below.
2. **A Bluetooth OBD-II adapter — the BLE kind** ("Bluetooth Low Energy" / "Bluetooth 4.0+"), not "Bluetooth Classic". Adapters from **Veepeak** or **Konnwei** advertising BLE support are known to work; any listing mentioning "OBD", "ELM", "VEEPEAK", "VLINK", "OBDLINK" or "KONNWEI" in its name should be found automatically by the board. Around £15–20.
3. **A microSD card**, 32GB or smaller, plus a card reader if your computer doesn't have a slot built in. Larger than 32GB needs an extra step (covered on your OS page).
4. **A USB cable that actually carries data, not just power.** Some cheap cables only charge — if your computer never sees the board, try a different cable first.
5. Something to power the board in the car — a spare phone charger into a 12V-to-USB adapter, or your car's own USB port, both work.

04 Setting up your WiFi & Telegram secrets

Every board needs to know your WiFi network name and password before it can connect to the internet, and — if you want phone alerts — a Telegram bot token. These live in **one file on your own computer**, never typed into a chat with anyone, AI or human. Do this step after installing the software (your OS section tells you exactly when).

1. In the project folder, go to `boards/cyd-s3-3p5/firmware/src/`. You'll see a file called `config.example.h`. **Make a copy of it in the same folder and rename the copy to `config.h`** (not `config.example.h` — the board only reads `config.h`).
2. Open `config.h` in VS Code (double-click the file).
3. Find these two lines and put your own WiFi network name and password between the quote marks:

```
#define WIFI_STA_SSID "Vitoi"
#define WIFI_STA_PASS "YOUR_WIFI_PASSWORD"
```

4. **Optional — Telegram alerts on your phone.** If you want a message when a regeneration starts or finishes:
 - Open Telegram, search for **@BotFather**, and send it `/newbot`. Follow its prompts (a name, then a username ending in "bot"). It replies with a long token — copy it.
 - Send your new bot any message (e.g. "hi") so it knows who you are.
 - In a browser, go to `https://api.telegram.org/bot<your token>/getUpdates` and look for `"chat":{"id":` — the number right after it is your chat ID.
 - Back in `config.h`, fill in `TG_BOT_TOKEN` and `TG_CHAT_ID` with these values.
 - Don't want Telegram? Leave both lines exactly as they are — the feature switches itself off.

KEEP THIS PRIVATE

Never share your `config.h` file, and never commit or paste its contents anywhere — it's listed in this project's `.gitignore` specifically so a normal "save my changes" step can't accidentally publish it.

05 Formatting the microSD card

The board needs the card formatted as **FAT32**. This matters because if the card is blank, unformatted, or in the wrong format, the board **will not warn you** — it silently falls back to its own tiny internal memory (room for a few minutes of driving) instead. Each OS section below has the exact steps for that computer.

06 Checking it's actually working

1. Power the board from USB. You should hear a short boot jingle and the screen should light up.
2. With no car connected yet, the screen shows **"NO LOGGER DATA"** — expected; the board is alive but hasn't found an OBD adapter yet.
3. Plug your Bluetooth OBD adapter into the car and turn the ignition to "on" (engine doesn't need to be running). Within a minute, the screen should show live numbers on a coloured background:
 - **Green** — everything normal, soot is low.
 - **Amber** — soot is building up, keep an eye on it.
 - **Red** — soot is close to the point where the car will regenerate on its own.
 - **"ACTIVE REGENERATION"** on a dark screen — it's burning the soot off now; don't switch the engine off until it finishes.
4. If WiFi is set up, the board also joins your home network and your phone can see a live dashboard. Your OS page shows how to find its address.

This is a hobby project, not a diagnostic tool — it reads numbers the car's engine computer already has and cannot tell you the filter is actually blocked or safe. Don't interact with the screen or your phone while driving.

Install the software and flash the board

This section covers two things only: getting the tools installed on your Windows computer, and getting the firmware onto the CYD 3.5" board. It's 100% point-and-click — no command line.

1. Install Visual Studio Code

1. Go to **code.visualstudio.com** in your browser and click **Download for Windows**.
2. Double-click the downloaded installer file (in your **Downloads** folder).
3. Click through the installer: accept the licence, then **Next** on every screen without changing anything.
4. Click **Install**, then **Finish**. VS Code opens on its own.

2. Install the PlatformIO extension

1. In VS Code, click the **Extensions** icon (four squares) in the left-hand strip.
2. Type **PlatformIO IDE** in the search box and click **Install** on the result published by PlatformIO.
3. Wait — this can take **several minutes** the first time. VS Code may ask to **reload**; let it. When done, an alien-head icon appears in the left strip.

3. Get the project files

If you have a repository URL: in VS Code press `Ctrl+Shift+P`, type **Git: Clone**, paste the URL, and pick a folder (e.g. Documents) to save it into.

If you were given a folder or ZIP file: right-click the ZIP in File Explorer, choose **Extract All...**, and pick a destination. You should end up with an `obd-esp32` folder containing a `boards` folder.

4. Open the correct project folder

This repo holds several boards' firmware, each in its own subfolder. Open the exact folder:

```
obd-esp32/boards/cyd-s3-3p5/firmware
```

Via **File** → **Open Folder...**, browse to that exact `firmware` subfolder and click **Select Folder**.

IMPORTANT

This project's `platformio.ini` deliberately pins an older tool version (`platform = espressif32@7.1.3`) because a newer one is known to crash this firmware's Bluetooth code. If anything offers to **update** the platform, decline it.

5. Set up your secrets

Do the `config.h` step now (section 4 above), then come back.

6. Format the microSD card as FAT32

1. Insert the card into your computer's card slot or a USB reader.
2. Open **File Explorer**, find the card under **This PC**.
3. Right-click the drive, choose **Format...**, set **File system** to **FAT32**, click **Start**.

If your card is larger than 32GB, Windows' Format dialog won't offer FAT32 at all — use a 32GB or smaller card, or a third-party FAT32 tool.

7. Connect the board

Plug in via a data-capable USB-C cable. This board has no separate USB-to-serial chip (no CP2102/CH340) — it uses the ESP32-S3's native USB connection, which Windows 10/11 normally recognises with no extra driver (*unverified on a genuinely fresh machine, but expected behaviour*). Check **Device Manager** → **Ports (COM & LPT)** for a new COM port.

8. Build and upload

In the blue toolbar at the **bottom** of VS Code: click the **checkmark (✓)** to Build, then the **right-arrow (→)** to Upload. Watch the terminal pane for a green `[SUCCESS]` line (red `[FAILED]` means something needs fixing). First build takes several minutes.

9. Open the Serial Monitor

Click the **plug icon** in the same toolbar to watch the board's own boot text — confirms the jingle and Bluetooth scan happened.

Troubleshooting (Windows)

PROBLEM	LIKELY CAUSE	WHAT TO TRY
No COM port appears	Charge-only cable, or no power	Swap USB cable and port.
COM port shows "Unknown device"	Driver mismatch (<i>unverified how often</i>)	Replug; check Windows Update; check board listing for driver notes.
"Access is denied" on upload	Another program has the port open	Close other serial monitors / Arduino IDE, retry.
Upload fails partway	Loose/poor cable	Try a shorter known-good cable, plug directly in (no hub).
Antivirus blocks upload	Some AV interferes with flashing tools (<i>unverified for this project</i>)	Temporarily pause real-time protection for the upload only.

Install the software and flash the board

This section covers installing the software and getting the firmware onto the board. Everything is clicking and typing into windows — no Terminal, except one optional step near the end that you can skip entirely.

1. Install Visual Studio Code

1. Go to **code.visualstudio.com**, click **Download for macOS**.
2. Double-click the downloaded `.zip` — it unzips automatically into `Visual Studio Code.app`.
3. Drag it onto your **Applications** folder, then open it from there.
4. The first time, macOS will warn "downloaded from the Internet" (this is Gatekeeper — normal for any downloaded app). Click **Open**.

2. Install the PlatformIO extension

1. Click the **Extensions** icon (four squares) in VS Code's left-hand strip.
2. Search **PlatformIO IDE**, click **Install** on the result published by PlatformIO.
3. Wait several minutes for the first-time download; let it reload if asked. An alien-head icon appears when done.

3. Get the project files

With a repository URL: use PlatformIO Home's **Clone Git Project** button, paste the URL, choose a save folder (e.g. Documents).

With a folder/ZIP: double-click the ZIP — macOS extracts it automatically — then move the resulting `obd-esp32` folder somewhere easy to find.

4. Set up your secrets

Do the `config.h` step now (section 4 earlier in this guide) before opening the project in the next step.

5. Open the right project folder

File → **Open Folder...**, navigate to:

```
obd-esp32/boards/cyd-s3-3p5/firmware
```

Select that exact `firmware` folder and click **Open**.

IMPORTANT

platformio.ini pins platform = espressif32@7.1.3 deliberately (a newer version crashes this firmware's Bluetooth code). If anything offers to "update" the platform, say no.

6. Format the microSD card

1. Insert the card, open **Disk Utility** (Cmd+Space, type "Disk Utility").
2. Select the card itself (not an indented volume), click **Erase**.
3. Set **Format** to **MS-DOS (FAT)** — macOS's name for FAT32 — and click **Erase**.

Cards over 32GB won't offer FAT32 in Disk Utility's normal window — use a 32GB-or-smaller card, or see the optional Terminal workaround at the end of the Mac page in the full docs.

7. Connect the board

Use a data-capable USB-C cable. No separate USB driver is needed (native USB connection) — look for it in PlatformIO's device list as something like `/dev/cu.usbmodemXXXX` (*exact appearance unverified on a fresh Mac*).

8. Build and upload

Bottom toolbar: **checkmark (✓)** = Build, **right-arrow (→)** = Upload. Watch for green **SUCCESS**. First build takes several minutes.

9. Open the Serial Monitor

Click the **plug icon** to watch boot text and confirm the jingle/Bluetooth scan.

Troubleshooting (Mac)

PROBLEM	LIKELY CAUSE	WHAT TO TRY
No device in PlatformIO's list	Charge-only cable	Swap to a data-capable cable.
Still nothing after swapping cable	Mac isn't seeing hardware at all	Check Apple menu → About This Mac → System Information → USB.
Upload fails partway	Loose connection, or port held by another app	Replug, close other serial monitors, retry.
Gatekeeper blocks something	Standard "downloaded from internet" warning	Right-click → Open, confirm once.
Unsigned extension / system-extension dialog	This board shouldn't need one — likely unrelated	Don't grant it just to "fix" this; check cable/port first.

Install the software and flash the board

Written for **Ubuntu with GNOME**, the most common beginner distro — other distros follow the same steps with different menus. Almost everything is point-and-click; there is exactly **one** unavoidable terminal command (step 7), explained in plain English, done once.

1. Install Visual Studio Code

Open **Ubuntu Software** (shopping-bag icon), search "**Visual Studio Code**", click **Install**. Other distros: download the `.deb` or `.rpm` from code.visualstudio.com and double-click it to open your package installer.

2. Install the PlatformIO extension

1. Click the **Extensions** icon (four squares) in VS Code's sidebar.
2. Search **PlatformIO IDE**, click **Install**.
3. Wait several minutes for the first-time download; let it reload if asked.

3. Get the project files

With a repository URL: use VS Code's **Source Control** icon → **Clone Repository**, paste the URL, pick a save folder.

With a folder/ZIP: right-click the ZIP in your file manager, choose **Extract Here**.

4. Open the right project folder

File → **Open Folder...**, navigate to:

```
obd-esp32/boards/cyd-s3-3p5/firmware
```

IMPORTANT

`platformio.ini` pins `platform = espressif32@7.1.3` on purpose (newer versions crash this firmware's Bluetooth code). Decline any offer to "update" it.

5. Set up your secrets

Do the `config.h` step now (section 4 earlier in this guide).

6. Format the microSD card as FAT32

Open the **Disks** app (pre-installed on Ubuntu), select your card, use the gear menu → **Format Partition...**, choose **FAT32**. Unlike Windows/macOS, Disks has no 32GB ceiling. **GParted** is an alternative if Disks isn't installed.

7. Fix serial port permissions (one-time)

By default, your user account can't access the board's serial device, which shows up as a "permission denied" error during upload. Fix it once:

1. Open a **Terminal** (Activities → type "Terminal").
2. Type exactly, then press Enter:

```
sudo usermod -a -G dialout $USER
```

This adds your account to the `dialout` group, which controls serial device access. It will ask for your login password (won't show on screen — that's normal). **Arch-based distros** use `uucp` instead of `dialout`.

3. **Log out and back in (or restart)** — the change only applies from your next login. This is the step people most often forget.

8. Connect the board

Use a data-capable USB-C cable.

9. Build and upload

Bottom toolbar: **checkmark (✓)** = Build, **right-arrow (→)** = Upload. Green `[SUCCESS]` means it worked; "Permission denied" on `/dev/ttyACM0` almost always means step 7 hasn't taken effect yet (log out/in and retry).

10. Open the Serial Monitor

Click the **plug icon** to watch the board boot and confirm the jingle/Bluetooth scan.

Troubleshooting (Linux)

PROBLEM	WHAT TO TRY
"Permission denied" on upload	Step 7 not done, or you haven't logged out/in since. Do that, retry.
No port appears at all	Try a different (data-capable) cable and USB port. (<i>unverified</i>) ModemManager/brltty are sometimes reported to interfere with serial devices on some distros — try cable/port first.
Upload fails partway	Unplug/replug the board and retry.

Extension install
hangs

Check your connection; close/reopen VS Code and retry.

This is a hobby project, not a diagnostic tool — it reads numbers the car's engine computer already reports and cannot tell you the filter is actually blocked or safe. Follow your vehicle's handbook and a qualified mechanic for anything safety-related. Not ready to build one yourself? Ask whoever gave you this guide about getting a board that's already set up and tested.